

Fixing Lunar Lander Touchdown Validation

<https://gemini.google.com/app/2d2d8f8e2ada1a3d>

User prompt: import math import random import pygame WIDTH, HEIGHT = 800, 600 STEP = 20 THRUST, ROTATE_SPEED, BURN_RATE, FUEL_MAX = 60.0, 2.4, 22.0, 300.0 MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25 FOOT = 12

```
def ship_color(fuel_ratio): """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default.""" pass
def on_landing(score): """Called after a successful landing with the points just earned; add fireworks or bonuses here.""" pass
def bonus_life_threshold(): """Return a score value at which the player earns an extra life, or None to disable bonus lives.""" pass
def make_terrain(): heights, y = [], random.randint(430, 520) for _ in range(WIDTH // STEP + 1): y = max(380, min(560, y + random.randint(-32, 32))) heights.append(y) pads = [] for start, length, mult in ((random.randint(2, 12), 4, 1), (random.randint(20, 35), 3, 3)): for i in range(start, start + length): heights[i] = heights[start] pads.append((start * STEP, (start + length - 1) * STEP, heights[start], mult)) return heights, pads
def ground_y(heights, x): x = max(0, min(WIDTH - 1, x)) i = int(x // STEP) t = (x - i * STEP) / STEP return heights[i] * (1 - t) + heights[i + 1] * t
def wrap_angle(angle): return (angle + math.pi) % math.tau - math.pi
class Game:
    def __init__(self): self.font = pygame.font.Font(None, 26)
    self.reset()
    def reset(self): self.level, self.score, self.lives = 1, 0, 3 self.bonus_earned = 0 self.new_round()
    def new_round(self): self.heights, self.pads = make_terrain() self.pos = pygame.Vector2(random.randint(100, 700), 70)
    self.vel = pygame.Vector2(random.uniform(-20, 20), 0) self.angle, self.fuel, self.thrusting = 0.0, FUEL_MAX, False
    self.state, self.message = "fly", ""
    def pad_under(self): for x1, x2, y, mult in self.pads: if x1 <= self.pos.x - 8 and self.pos.x + 8 <= x2: return (x1, x2, y, mult) return None
    def touchdown(self): pad = self.pad_under() angle = wrap_angle(self.angle) if pad and abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE: earned = int((100 + self.fuel) * pad[3]) self.score += earned self.state, self.message = "landed", f"Perfect landing! +{earned} (Space = next level)" on_landing(earned) return self.lives -= 1 self.state = "crashed" if pad is None: reason = "missed the pad" elif abs(angle) > MAX_ANGLE: reason = "bad angle" else: reason = "too fast" self.message = f"Crashed: {reason}! " + ("Space = retry" if self.lives > 0 else "Game over - R = restart")
    def update(self, dt, keys): if self.state != "fly": return threshold = bonus_life_threshold() if threshold and self.score // threshold > self.bonus_earned: self.bonus_earned = self.score // threshold self.lives += 1 self.angle += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * ROTATE_SPEED * dt gravity = pygame.Vector2(0, 16 + 2 * self.level) self.thrusting = bool(keys[pygame.K_UP]) and self.fuel > 0 acceleration = gravity if self.thrusting: acceleration = gravity + pygame.Vector2(math.sin(self.angle), -math.cos(self.angle)) * THRUST self.fuel = max(0.0, self.fuel - BURN_RATE * dt) self.vel += acceleration * dt self.pos += self.vel * dt self.pos.x %= WIDTH self.pos.y = max(-200, self.pos.y) if self.pos.y + FOOT >= ground_y(self.heights, self.pos.x): self.touchdown()
    def ship_points(self): cos, sin = math.cos(self.angle), math.sin(self.angle) local = [(0, -16), (10, 10), (-10, 10)] return [(self.pos.x + x * cos - y * sin, self.pos.y + x * sin + y * cos) for x, y in local]
    def draw(self, screen): screen.fill((8, 8, 20)) points = [(i * STEP, h) for i, h in enumerate(self.heights)] pygame.draw.polygon(screen, (70, 70, 85), points + [(WIDTH, HEIGHT), (0, HEIGHT)]) pygame.draw.lines(screen, (190, 190, 200), False, points, 2) for x1, x2, y, mult in self.pads: pygame.draw.line(screen, (90, 230, 120), (x1, y), (x2, y), 5) label = self.font.render(f"x{mult}", True, (90, 230, 120)) screen.blit(label, label.get_rect(midtop=((x1 + x2) / 2, y + 8))) if self.state != "crashed": if self.thrusting: cos, sin = math.cos(self.angle), math.sin(self.angle) flame = [(self.pos.x - 5 * cos - 10 * sin, self.pos.y - 5 * sin + 10 * cos), (self.pos.x + 5 * cos - 10 * sin, self.pos.y + 5 * sin + 10 * cos), (self.pos.x - 22 * sin, self.pos.y + 22 * cos)] pygame.draw.polygon(screen, (255, 170, 40), flame) color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (230, 230, 240) pygame.draw.polygon(screen, color, self.ship_points()) else: pygame.draw.circle(screen, (255, 120, 40), self.pos, 24, 3) ok_x = abs(self.vel.x) <= MAX_SPEED_X ok_y = abs(self.vel.y) <= MAX_SPEED_Y ok_a = abs(wrap_angle(self.angle)) <= MAX_ANGLE good, bad = (120, 240, 140), (250, 110, 100) lines = [ (f"Fuel {self.fuel:5.0f}", (240, 240, 240)), (f"Vx {self.vel.x:6.1f}", good if ok_x else bad), (f"Vy {self.vel.y:6.1f}", good if ok_y else bad), (f"Angle {math.degrees(wrap_angle(self.angle)):5.0f}", good if ok_a else bad), (f"Score {self.score} Lives {self.lives} Level {self.level}", (240, 240, 240)), ] for i, (text, color) in enumerate(lines): screen.blit(self.font.render(text, True, color), (10, 8 + i * 22)) if self.message: label = self.font.render(self.message, True, (255, 255, 120)) screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3)))
    def main(): pygame.init() screen = pygame.display.set_mode((WIDTH, HEIGHT)) pygame.display.set_caption("Lunar Lander") clock = pygame.time.Clock() game = Game() running = True while running: dt = min(clock.tick(60) / 1000, 0.05) for event in pygame.event.get(): if event.type == pygame.QUIT: running = False elif event.type == pygame.KEYDOWN: if event.key == pygame.K_r: game.reset() elif event.key == pygame.K_SPACE and game.state == "landed": game.level += 1 game.new_round() elif event.key == pygame.K_SPACE and game.state == "crashed" and game.lives > 0: game.new_round() game.update(dt, pygame.key.get_pressed()) game.draw(screen) pygame.display.flip() pygame.quit() if __name__ == "__main__": main()
This is the code for my lunar lander game, I need to do following task one by one, for each task I will prompt you with that particular task, now for the first task 1. Fix the touchdown validation bug > A safe landing is supposed to require the ship to be over a pad, descending slowly, roughly level, *and* not moving sideways too fast. In the current build, a ship can slam down sideways — well beyond `MAX_SPEED_X` — on a pad at a safe vertical speed and angle, and it still counts as a perfect landing. Check the condition in `touchdown()` against all three constants it defines near the top of the file (`MAX_SPEED_X`, `MAX_SPEED_Y`, `MAX_ANGLE`), and see which one it never actually checks.
```

Response: In Game.touchdown(), the current validation condition only checks self.vel.y and angle:

```
if pad and abs(self.vel.y) <= MAX_SPEED_Y and abs(angle) <= MAX_ANGLE:
```

It completely ignores horizontal speed (`self.vel.x`), allowing the ship to land safely regardless of how fast it moves sideways.

Fix

Update `touchdown()` to include the `abs(self.vel.x) <= MAX_SPEED_X` check and update the crash reason message to reflect horizontal velocity violations:

```
def touchdown(self):
    pad = self.pad_under()
    angle = wrap_angle(self.angle)
    if (
        pad
        and abs(self.vel.x) <= MAX_SPEED_X
        and abs(self.vel.y) <= MAX_SPEED_Y
        and abs(angle) <= MAX_ANGLE
    ):
        earned = int((100 + self.fuel) * pad[3])
        self.score += earned
        self.state, self.message = (
            "landed",
            f"Perfect landing! +{earned} (Space = next level)",
        )
        on_landing(earned)
        return

    self.lives -= 1
    self.state = "crashed"
    if pad is None:
        reason = "missed the pad"
    elif abs(angle) > MAX_ANGLE:
        reason = "bad angle"
    elif abs(self.vel.x) > MAX_SPEED_X:
        reason = "too fast horizontally"
    else:
        reason = "too fast vertically"

    self.message = (
        f"Crashed: {reason}! "
        + ("Space = retry" if self.lives > 0 else "Game over - R = restart")
    )
```

Ready for task 2!

User prompt: Task 2: Implement `ship_color(fuel_ratio)` > Called once per frame in `draw`, as `color = ship_color(max(0.0, self.fuel) / FUEL_MAX)` or `(230, 230, 240)`. It receives the remaining fuel as a fraction from `0.0` (empty) to `1.0` (full) and should return an `(r, g, b)` hull color, or `None` to keep the default. Idea: shift the hull toward red as fuel runs low.

Response: Here is the implementation for `ship_color(fuel_ratio)`. It smoothly interpolates the hull color from white/bright grey (230, 230, 240) at full fuel (1.0) down to a bright alert red (255, 60, 60) when out of fuel (0.0).

Implementation

```
def ship_color(fuel_ratio):
    """Return an (r, g, b) hull colour for the given fuel ratio (1.0 = full), or None for the default."""
    # Ensure fuel_ratio stays within valid bounds [0.0, 1.0]
    ratio = max(0.0, min(1.0, fuel_ratio))

    # Color at full fuel (ratio = 1.0): default light grey/white
    r_full, g_full, b_full = 230, 230, 240

    # Color at empty fuel (ratio = 0.0): warning red
    r_empty, g_empty, b_empty = 255, 60, 60

    # Linear interpolation between full and empty states
```

```

r = int(r_empty + (r_full - r_empty) * ratio)
g = int(g_empty + (g_full - g_empty) * ratio)
b = int(b_empty + (b_full - b_empty) * ratio)

return (r, g, b)

```

Ready for Task 3 whenever you're set!

User prompt: Task 3: Implement `on\_landing(score)` > Called from `touchdown()` immediately after a successful landing's points are added to the score. It receives the number of points just earned from that landing. Its return value is ignored. Idea: a fireworks effect, or a distinct sound for a high-multiplier pad.

Response: Here is an implementation of on_landing(score) that adds a visual fireworks particle explosion directly on the landing pad when you touch down.

Since Game manages drawing and state updates, we can store global particle instances or trigger a particle burst at the ship's current location/pad.

Implementation

To support the visual effect smoothly, update on_landing(score) to generate a list of active particle effects, and modify Game.draw() to render them:

```

# Place this at module level (near top of file with other globals)
fireworks = []

def on_landing(score):
    """Called after a successful landing with the points just earned; add fireworks or bonuses here."""
    # Spawn colorful fireworks particle burst
    # Scale particle count based on points earned (more points = bigger burst)
    num_particles = min(120, max(40, score // 3))

    for _ in range(num_particles):
        angle = random.uniform(0, math.tau)
        speed = random.uniform(30, 180)
        velocity = pygame.Vector2(
            math.cos(angle) * speed, math.sin(angle) * speed - 40
        )
        color = random.choice(
            [
                (255, 215, 0), # Gold
                (255, 80, 80), # Red
                (100, 255, 120), # Bright Green
                (100, 200, 255), # Cyan
                (255, 150, 255), # Magenta
            ]
        )
        fireworks.append(
            {
                "pos": pygame.Vector2(
                    game_instance.pos.x if "game_instance" in globals() else 400,
                    game_instance.pos.y if "game_instance" in globals() else 300,
                ),
                "vel": velocity,
                "color": color,
                "lifetime": random.uniform(0.6, 1.4),
                "age": 0.0,
            }
        )

```

To update and render these fireworks in your main loop, update Game.draw(screen) (or update / draw loop):

```

# Add this snippet inside Game.draw(screen) right before blitting telemetry lines:
global fireworks

```

```

dt_frame = 1 / 60.0 # Approx dt for particle movement
remaining = []
for p in fireworks:
    p["age"] += dt_frame
    if p["age"] < p["lifetime"]:
        p["vel"].y += 120 * dt_frame # Gravity on particles
        p["pos"] += p["vel"] * dt_frame
        alpha_ratio = 1.0 - (p["age"] / p["lifetime"])
        # Fade color
        r, g, b = p["color"]
        fade_color = (
            int(r * alpha_ratio),
            int(g * alpha_ratio),
            int(b * alpha_ratio),
        )
        pygame.draw.circle(
            screen, fade_color, (int(p["pos"].x), int(p["pos"].y)), 3
        )
        remaining.append(p)
fireworks = remaining

```

And store `game_instance = self` inside `Game.__init__` so `on_landing` knows where the lander landed:

```

class Game:

    def __init__(self):
        global game_instance
        game_instance = self
        self.font = pygame.font.Font(None, 26)
        self.reset()

```

Ready for Task 4 whenever you are!

User prompt: Task 4: Implement `bonus_life_threshold()` > Called every frame in `update()`. It takes no arguments and should return an integer score value, or `None` to disable bonus lives entirely. Whenever the score crosses a multiple of that value for the first time, one life is awarded automatically — the bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose the threshold. Idea: return `1500`.

Response: Here is the implementation for `bonus_life_threshold()`:

```

def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
    return 1500

```

How it works with the existing codebase

In `Game.update()`, the game checks:

```

threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1

```

By returning `1500`, the player automatically earns an extra life at 1,500 points, 3,000 points, 4,500 points, and so on.

Let me know if you have any other tasks or features you'd like to add!

User prompt: can you give me the entire code of this without any comments and it should include all the four tasks provided

Response: Here is the complete, working Python script for the Lunar Lander game with all four tasks fully integrated and all comments removed.

```
import math
```

```
import random
import pygame

WIDTH, HEIGHT = 800, 600
STEP = 20
THRUST, ROTATE_SPEED, BURN_RATE, FUEL_MAX = 60.0, 2.4, 22.0, 300.0
MAX_SPEED_X, MAX_SPEED_Y, MAX_ANGLE = 25.0, 40.0, 0.25
FOOT = 12

game_instance = None
fireworks = []

def ship_color(fuel_ratio):
    ratio = max(0.0, min(1.0, fuel_ratio))
    r = int(255 + (230 - 255) * ratio)
    g = int(60 + (230 - 60) * ratio)
    b = int(60 + (240 - 60) * ratio)
    return (r, g, b)

def on_landing(score):
    global fireworks
    num_particles = min(120, max(40, score // 3))
    x = game_instance.pos.x if game_instance else 400
    y = game_instance.pos.y if game_instance else 300
    for _ in range(num_particles):
        angle = random.uniform(0, math.tau)
        speed = random.uniform(30, 180)
        velocity = pygame.Vector2(
            math.cos(angle) * speed, math.sin(angle) * speed - 40
        )
        color = random.choice(
            [
                (255, 215, 0),
                (255, 80, 80),
                (100, 255, 120),
                (100, 200, 255),
                (255, 150, 255),
            ]
        )
        fireworks.append(
            {
                "pos": pygame.Vector2(x, y),
                "vel": velocity,
                "color": color,
                "lifetime": random.uniform(0.6, 1.4),
                "age": 0.0,
            }
        )

def bonus_life_threshold():
    return 1500

def make_terrain():
    heights, y = [], random.randint(430, 520)
    for _ in range(WIDTH // STEP + 1):
        y = max(380, min(560, y + random.randint(-32, 32)))
        heights.append(y)
    pads = []
    for start, length, mult in (
        (random.randint(2, 12), 4, 1),
        (random.randint(20, 35), 3, 3),
    ):
        pass
```

```
        for i in range(start, start + length):
            heights[i] = heights[start]
        pads.append(
            (start * STEP, (start + length - 1) * STEP, heights[start], mult)
        )
    return heights, pads

def ground_y(heights, x):
    x = max(0, min(WIDTH - 1, x))
    i = int(x // STEP)
    t = (x - i * STEP) / STEP
    return heights[i] * (1 - t) + heights[i + 1] * t

def wrap_angle(angle):
    return (angle + math.pi) % math.tau - math.pi

class Game:

    def __init__(self):
        global game_instance
        game_instance = self
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.level, self.score, self.lives = 1, 0, 3
        self.bonus_awarded = 0
        self.new_round()

    def new_round(self):
        self.heights, self.pads = make_terrain()
        self.pos = pygame.Vector2(random.randint(100, 700), 70)
        self.vel = pygame.Vector2(random.uniform(-20, 20), 0)
        self.angle, self.fuel, self.thrusting = 0.0, FUEL_MAX, False
        self.state, self.message = "fly", ""

    def pad_under(self):
        for x1, x2, y, mult in self.pads:
            if x1 <= self.pos.x - 8 and self.pos.x + 8 <= x2:
                return (x1, x2, y, mult)
        return None

    def touchdown(self):
        pad = self.pad_under()
        angle = wrap_angle(self.angle)
        if (
            pad
            and abs(self.vel.x) <= MAX_SPEED_X
            and abs(self.vel.y) <= MAX_SPEED_Y
            and abs(angle) <= MAX_ANGLE
        ):
            earned = int((100 + self.fuel) * pad[3])
            self.score += earned
            self.state, self.message = (
                "landed",
                f"Perfect landing! +{earned} (Space = next level)",
            )
            on_landing(earned)
            return
        self.lives -= 1
        self.state = "crashed"
        if pad is None:
            reason = "missed the pad"
```

```

elif abs(angle) > MAX_ANGLE:
    reason = "bad angle"
elif abs(self.vel.x) > MAX_SPEED_X:
    reason = "too fast horizontally"
else:
    reason = "too fast vertically"
self.message = (
    f"Crashed: {reason}! "
    + ("Space = retry" if self.lives > 0 else "Game over - R = restart")
)

def update(self, dt, keys):
    if self.state != "fly":
        return
    threshold = bonus_life_threshold()
    if threshold and self.score // threshold > self.bonus_awarded:
        self.bonus_awarded = self.score // threshold
        self.lives += 1
    self.angle += (
        (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * ROTATE_SPEED * dt
    )
    gravity = pygame.Vector2(0, 16 + 2 * self.level)
    self.thrusting = bool(keys[pygame.K_UP]) and self.fuel > 0
    acceleration = gravity
    if self.thrusting:
        acceleration = (
            gravity
            + pygame.Vector2(
                math.sin(self.angle), -math.cos(self.angle)
            )
            * THRUST
        )
        self.fuel = max(0.0, self.fuel - BURN_RATE * dt)
    self.vel += acceleration * dt
    self.pos += self.vel * dt
    self.pos.x %= WIDTH
    self.pos.y = max(-200, self.pos.y)
    if self.pos.y + FOOT >= ground_y(self.heights, self.pos.x):
        self.touchdown()

def ship_points(self):
    cos, sin = math.cos(self.angle), math.sin(self.angle)
    local = [(0, -16), (10, 10), (-10, 10)]
    return [
        (self.pos.x + x * cos - y * sin, self.pos.y + x * sin + y * cos)
        for x, y in local
    ]

def draw(self, screen):
    global fireworks
    screen.fill((8, 8, 20))
    points = [(i * STEP, h) for i, h in enumerate(self.heights)]
    pygame.draw.polygon(
        screen, (70, 70, 85), points + [(WIDTH, HEIGHT), (0, HEIGHT)]
    )
    pygame.draw.lines(screen, (190, 190, 200), False, points, 2)
    for x1, x2, y, mult in self.pads:
        pygame.draw.line(screen, (90, 230, 120), (x1, y), (x2, y), 5)
        label = self.font.render(f"x{mult}", True, (90, 230, 120))
        screen.blit(label, label.get_rect(midtop=((x1 + x2) / 2, y + 8)))

    remaining = []
    dt_frame = 1 / 60.0
    for p in fireworks:
        p["age"] += dt_frame
        if p["age"] < p["lifetime"]:
```

```

        p["vel"].y += 120 * dt_frame
        p["pos"] += p["vel"] * dt_frame
        alpha_ratio = 1.0 - (p["age"] / p["lifetime"])
        r, g, b = p["color"]
        fade_color = (
            int(r * alpha_ratio),
            int(g * alpha_ratio),
            int(b * alpha_ratio),
        )
        pygame.draw.circle(
            screen, fade_color, (int(p["pos"].x), int(p["pos"].y)), 3
        )
        remaining.append(p)
    fireworks = remaining

    if self.state != "crashed":
        if self.thrusting:
            cos, sin = math.cos(self.angle), math.sin(self.angle)
            flame = [
                (
                    self.pos.x - 5 * cos - 10 * sin,
                    self.pos.y - 5 * sin + 10 * cos,
                ),
                (
                    self.pos.x + 5 * cos - 10 * sin,
                    self.pos.y + 5 * sin + 10 * cos,
                ),
                (self.pos.x - 22 * sin, self.pos.y + 22 * cos),
            ]
            pygame.draw.polygon(screen, (255, 170, 40), flame)
            color = ship_color(max(0.0, self.fuel) / FUEL_MAX) or (
                230,
                230,
                240,
            )
            pygame.draw.polygon(screen, color, self.ship_points())
        else:
            pygame.draw.circle(screen, (255, 120, 40), self.pos, 24, 3)
            ok_x = abs(self.vel.x) <= MAX_SPEED_X
            ok_y = abs(self.vel.y) <= MAX_SPEED_Y
            ok_a = abs(wrap_angle(self.angle)) <= MAX_ANGLE
            good, bad = (120, 240, 140), (250, 110, 100)
            lines = [
                (f"Fuel {self.fuel:5.0f}", (240, 240, 240)),
                (f"Vx {self.vel.x:6.1f}", good if ok_x else bad),
                (f"Vy {self.vel.y:6.1f}", good if ok_y else bad),
                (
                    f"Angle {math.degrees(wrap_angle(self.angle)):5.0f}",
                    good if ok_a else bad,
                ),
                (
                    f"Score {self.score} Lives {self.lives} Level {self.level}",
                    (240, 240, 240),
                ),
            ]
            for i, (text, color) in enumerate(lines):
                screen.blit(self.font.render(text, True, color), (10, 8 + i * 22))
            if self.message:
                label = self.font.render(self.message, True, (255, 255, 120))
                screen.blit(
                    label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3))
                )

def main():
    pygame.init()

```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Lunar Lander")
clock = pygame.time.Clock()
game = Game()
running = True
while running:
    dt = min(clock.tick(60) / 1000, 0.05)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                game.reset()
            elif event.key == pygame.K_SPACE and game.state == "landed":
                game.level += 1
                game.new_round()
            elif (
                event.key == pygame.K_SPACE
                and game.state == "crashed"
                and game.lives > 0
            ):
                game.new_round()
    game.update(dt, pygame.key.get_pressed())
    game.draw(screen)
    pygame.display.flip()
pygame.quit()

if __name__ == "__main__":
    main()
```
